Case Study 1: RISC vs. CISC — Choosing a Processor

Student Name: Hardik Aggarwal
UID: 25BAI10001
Class: 25BAI601
Subject: Computer Organisation and Architecture

Scenario and Problem

A technology company is working on two computing systems, one based on a RISC architecture and the other on a CISC architecture. The team has to pick the instruction-set design that fits a low-power embedded device better. So the task is to compare both architectures and explain which one should be preferred for this use case.

Case Study Questions Answers

1. What is RISC? What is CISC?

RISC stands for Reduced Instruction Set Computer. The idea behind it is to give the processor only a small number of simple instructions, and to make each of them quick, ideally one clock cycle. If a bigger job has to be done, it is just broken into several of these simple instructions. Some points that usually go with RISC:

- Only LOAD and STORE instructions are allowed to access memory. Everything else works on registers.
- Instructions have the same length, normally 32 bits.
- There are plenty of general-purpose registers, so the processor does not have to go to memory again and again.
- The control unit is usually hardwired instead of microprogrammed.

ARM, RISC-V and MIPS are common examples.

CISC stands for Complex Instruction Set Computer, and it takes the opposite route. The processor has a big set of instructions, and one instruction can carry out a whole multi-step operation, for example fetching a value from memory, adding to it and writing it back. This was originally done to keep programs short, because memory was costly and small in the early days. Typical features of CISC are:

- A large number of instructions, including special ones for strings and blocks of data.
- Instructions of different lengths, from one byte to several bytes.
- Arithmetic can be done directly on data sitting in memory.
- Complex instructions are handled with microcode inside the control unit.

The Intel x86 family and the old VAX and IBM System/360 machines fall in this category.

To put it simply, RISC keeps the hardware simple and leaves more work to the compiler, while CISC makes the hardware do more so that programs become shorter.

2. Why do many RISC instruction sets use fixed-length instructions?

When all instructions have the same size, say 32 bits, a lot of things in the processor become easier. The main reasons are given below.

Fetching is easy

The processor already knows that the next instruction starts 4 bytes after the current one, so the program counter only has to add a constant. It does not need to look at the instruction first to find out where it ends.

Decoding is fast

The opcode and the register fields stay at the same bit positions in every instruction. Because of this the decoder can start reading the registers while it is still figuring out the opcode, and it needs less hardware.

Pipelining works better

In a pipeline, the fetch stage should keep bringing in a new instruction every cycle. With variable lengths it may have to wait to know how long the current instruction is, which causes gaps. Fixed length avoids that, and it also makes it easier for superscalar processors to look at several instructions together.

Memory alignment

Aligned 32-bit instructions fit neatly in cache lines, so an instruction never gets split across two blocks.

Drawback

The downside is that code gets a bit bigger, since even a simple instruction takes the full 32 bits. To reduce this, some designs such as ARM Thumb and the compressed extension of RISC-V also provide a 16-bit format for common instructions.

3. How does instruction complexity influence instruction decoding?

Decoding is the stage where the control unit reads the instruction and finds out what operation is needed, which registers are involved and where the operands are. If the instructions are complicated, this stage naturally gets harder and slower.

With simple instructions there are only a few formats, so the decoder just has to look at a few fixed fields. Such a decoder can be made using plain hardwired logic, which is quick. With complex instructions the decoder has to deal with many formats, different lengths and various addressing modes. It must first work out the length of the instruction and then decode the remaining part. A complex instruction may also need several internal steps, so the control unit has to generate a whole sequence of signals, and this is normally done with microcode stored in a control store. Reading from the control store takes extra time, and the bigger decoder also uses more chip area and more power.

In RISC, decoding usually fits in a single pipeline stage. In CISC it may need more than one stage. Modern x86 processors actually first convert each complex instruction into smaller micro-operations and then execute those, which adds one more step. So higher complexity means a

slower, larger and more power-consuming decoder, and this is a big reason why RISC is preferred where energy is limited.

4. Compare the typical addressing-mode approaches in RISC and CISC.

An addressing mode is simply the way an instruction specifies where its operand is, whether in a register, inside the instruction itself or at some place in memory.

In RISC

RISC processors keep the modes to a minimum, mostly register, immediate and base plus offset. Since only LOAD and STORE use memory, the memory addressing is limited to those two instructions, and all arithmetic is done between registers. The address calculation is easy and fits in one pipeline stage.

In CISC

CISC processors offer many more modes, such as direct, register indirect, indexed, based-indexed and auto-increment or decrement, and some even allow memory-indirect addressing. As most instructions can read operands from memory directly, the programs need fewer instructions. However, a few of these modes need more than one memory access just to get one operand, so the execution time is harder to predict and the hardware is more complicated.

Comparison

	RISC	**CISC**
No. of modes	Very few, around 3 or 4	A lot, often more than 10
Typical modes	Register, immediate, base + offset	Direct, indirect, indexed, auto-increment, memory-to-memory
Memory access	Only LOAD and STORE can touch memory	Most instructions can use memory operands
Address calculation	Quick, one simple step	Can take several steps and extra memory reads
Hardware	Simple and easy to pipeline	Complicated control logic
Code length	More instructions needed	Fewer instructions needed

So CISC addressing is more flexible and gives shorter code, whereas RISC addressing is simple, predictable and easier to make fast. For a low-power device, the simple approach of RISC is better because it needs less hardware.

5. Can a CISC processor outperform a RISC processor? Explain with one reason.

Yes, it can. How fast a processor is depends on many things like the type of program, the compiler, the fabrication technology and how well the chip is designed, so the instruction set alone does not decide it.

One reason is that a single CISC instruction can replace a long sequence of RISC instructions. For instance, copying a block of memory on x86 can be done with one instruction, REP MOVSB, but a RISC processor needs a loop that loads a byte, stores it, updates two pointers, decrements a counter and branches back. In such a case the CISC processor fetches fewer

instructions, uses less instruction-memory bandwidth, and its smaller code fits better in the instruction cache. The loop overhead is also handled inside the hardware.

It is also worth knowing that modern x86 chips break complex instructions into RISC-like micro-operations and use deep pipelines, large caches and out-of-order execution. That is one of the reasons they are still very strong in desktops and servers. For a low-power embedded device, though, the simpler RISC design usually remains the more sensible choice.